소프트웨어공학 강의노트 — 디자인 패턴 입문

※ 본 문서는 AKAI 시연을 위해 팀 타닥타닥이 직접 작성한 학습 정리 자료입니다.

가온대학교 소프트웨어학과 · 3 학년 전공선택 · 7 주차

1. 디자인 패턴이란

디자인 패턴은 소프트웨어 설계에서 반복해서 나타나는 문제와 그 해결 방법을 이름 붙여 정리한 것이다. 패턴은 완성된 코드가 아니라 설계의 틀이며, 상황에 맞게 바꾸어 적용한다.

디자인 패턴을 쓰는 이유는 세 가지다.

1. 의사소통: "여기는 Strategy 로 하자"처럼 짧은 말로 설계 의도를 전달할 수 있다.
2. 검증된 해법: 많은 개발자가 이미 겪은 문제를 같은 방식으로 풀어서, 시행착오를 줄인다.
3. 변경에 강한 구조: 자주 바뀌는 부분을 분리해 두어, 변경이 생겨도 고칠 곳이 적다.

패턴을 고를 때는 "이 시스템에서 무엇이 자주 바뀌는가"를 먼저 묻는다. 바뀌는 것을 찾으면 그것을 감싸는 패턴이 자연스럽게 정해진다.

2. 패턴의 분류

디자인 패턴은 목적에 따라 세 가지로 나눈다. 대표적인 분류는 1994 년에 출판된 GoF(Gang of Four)의 책에서 정리한 23 개 패턴이다.

분류	다루는 문제	대표 패턴
생성 패턴	객체를 어떻게 만들 것인가	Factory Method, Abstract Factory, Singleton, Builder
구조 패턴	클래스와 객체를 어떻게 조합할 것인가	Adapter, Decorator, Proxy, Composite
행위 패턴	객체들이 어떻게 책임을 나누고 협력할 것인가	Strategy, State, Observer, Template Method, Command

3. Strategy 패턴

3.1 문제

같은 일을 하는 방법이 여러 가지이고, 실행 중에 방법을 바꾸어야 하는 경우가 있다. 예를 들어 할인 계산 방식이 "정액 할인", "정률 할인", "할인 없음"으로 나뉘고 계속 늘어난다고 하자. if-else 로 나누면 새 방식을 추가할 때마다 기존 코드를 고쳐야 한다.

3.2 해법

알고리즘을 인터페이스로 정의하고, 방식마다 구현 클래스를 만든다. 사용하는 쪽(Context)은 인터페이스만 알고, 실제 구현은 외부에서 넣어 준다.

- **Strategy**: 알고리즘의 공통 인터페이스 (예: DiscountPolicy)
- **ConcreteStrategy**: 구체적인 알고리즘 (예: FixedDiscount, RateDiscount)
- **Context**: Strategy 를 사용하는 객체 (예: OrderService)

3.3 효과

새 할인 방식은 클래스 하나를 추가하는 것으로 끝나고 기존 코드는 고치지 않는다. 이는 개방-폐쇄 원칙(OCP)을 따르는 구조다. 단점은 클래스 수가 늘어나고, 사용하는 쪽이 어떤 전략이 있는지 알아야 한다는 점이다.

4. State 패턴

4.1 문제

객체의 상태에 따라 같은 요청에 대한 동작이 달라지는 경우가 있다. 예를 들어 주문은 "접수", "결제 완료", "배송 중", "배송 완료", "취소" 상태를 가지며, "취소" 요청은 접수와 결제 완료 상태에서만 허용된다. 모든 메서드 안에 상태를 확인하는 조건문을 넣으면 코드가 복잡해진다.

4.2 해법

상태마다 클래스를 만들고, 상태별로 허용되는 동작과 다음 상태로의 전이를 그 클래스 안에 둔다. 객체는 현재 상태 객체에게 요청을 넘긴다.

4.3 Strategy 와의 차이

두 패턴은 구조가 비슷하지만 의도가 다르다. Strategy 는 사용하는 쪽이 알고리즘을 골라서 넣고, State 는 상태 객체가 스스로 다음 상태로 바꾼다. 즉 Strategy 는 "어떻게 할지"를 바꾸고, State 는 "지금 무엇을 할 수 있는지"를 바꾼다.

5. Factory Method 패턴

5.1 문제

어떤 클래스의 객체를 만들지가 상황에 따라 달라지는데, 생성 코드가 여러 곳에 흩어져 있으면 새 종류를 추가할 때 모든 곳을 고쳐야 한다. 예를 들어 파일 형식(PDF, TXT, DOCX)에 따라 다른 파서 객체를 만들어야 하는 경우다.

5.2 해법

객체 생성을 담당하는 메서드를 따로 두고, 어떤 클래스를 만들지는 하위 클래스나 생성 메서드가 결정하게 한다. 사용하는 쪽은 생성된 객체의 공통 인터페이스만 사용한다.

5.3 효과

생성 로직이 한 곳에 모여 관리가 쉬워지고, 새 종류를 추가해도 사용하는 코드는 바뀌지 않는다.

6. Adapter 패턴

6.1 문제

이미 있는 클래스나 외부 라이브러리를 쓰고 싶은데, 인터페이스가 우리 시스템과 맞지 않는 경우가 있다. 예를 들어 외부 AI 서비스 두 곳의 호출 방식이 서로 다르면, 서비스를 바꿀 때마다 호출하는 코드 전체를 고쳐야 한다.

6.2 해법

우리 시스템이 원하는 인터페이스(Target)를 정하고, 외부 클래스(Adaptee)를 감싸서 그 인터페이스에 맞추는 Adapter 클래스를 만든다. 전원 플러그 변환기와 같은 역할이다.

6.3 효과

외부 라이브러리를 바꾸어도 Adapter 만 새로 만들면 된다. 의존 역전 원칙(DIP)과 함께 쓰면 상위 모듈이 외부 라이브러리에 직접 의존하지 않게 된다.

7. 패턴 적용 시 주의할 점

- 패턴은 문제가 있을 때 쓴다. 바뀔 가능성이 없는 부분에 패턴을 넣으면 구조만 복잡해진다.
- 처음부터 모든 패턴을 넣으려 하지 말고, 변경이 실제로 생기거나 생길 것이 분명할 때 리팩터링으로 도입한다.
- 패턴 이름을 클래스 이름에 넣으면(PdfParserFactory, PaymentAdapter) 설계 의도가 코드에 드러난다.

8. 확인 문제

1. Strategy 와 State 패턴의 차이를 의도 관점에서 설명하시오.
2. 외부 라이브러리 교체에 대비하려면 어떤 패턴이 적합한가?
3. 패턴을 고를 때 먼저 던져야 할 질문은 무엇인가?
4. GoF 가 분류한 세 가지 패턴 종류를 쓰시오.